



Rustman

Client API · Rust · Tauri · Svelte

Il client API con workflow visuali

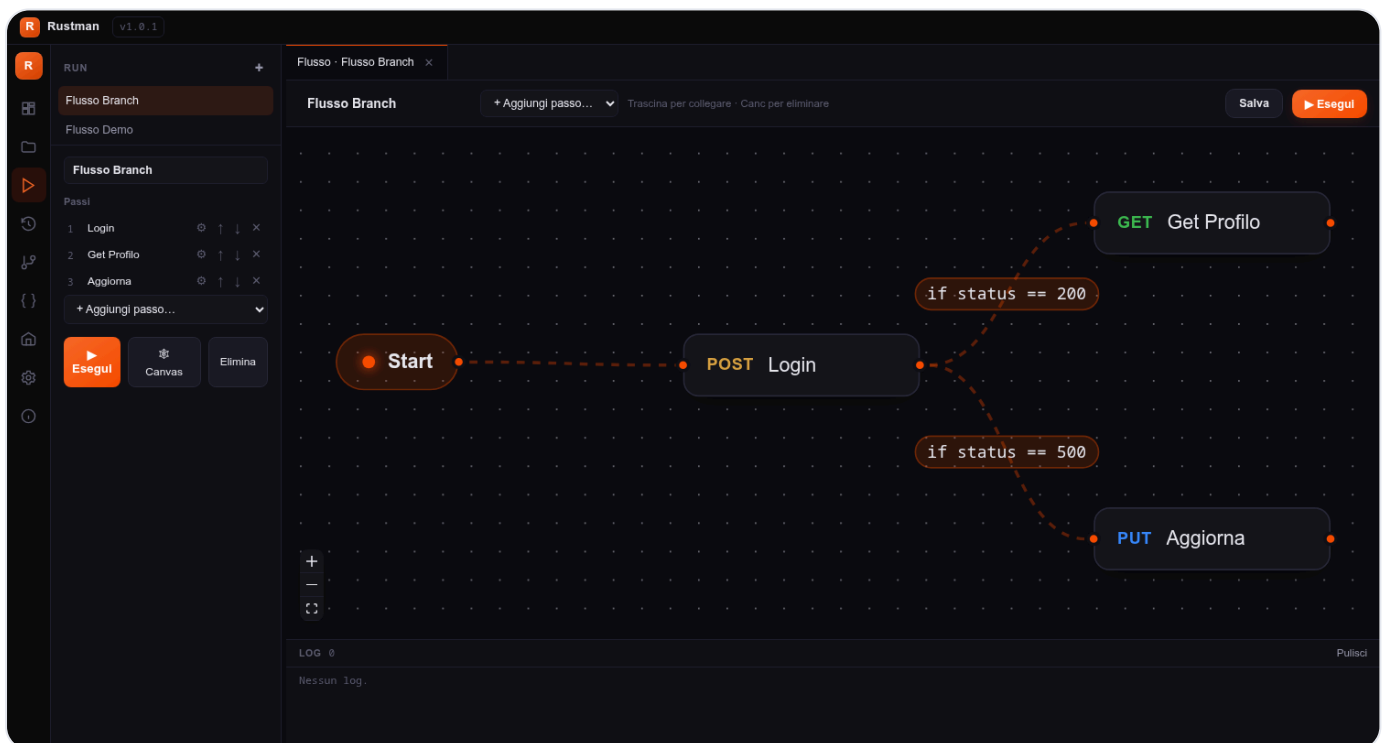
Ispirato a Postman, scritto in Rust. Veloce, leggero, con Git integrato e un canvas a nodi per costruire i test d'integrazione come un grafo — con rami condizionali e merge reali.

v1.1.0

Desktop & Web

Linux · Windows · macOS

Open source · MIT



github.com/russus87/rustman · Brochure & Documentazione

Un client API completo, in un binario nativo

Rustman unisce la comodità di un client API moderno alla velocità di Rust e Tauri. Le collezioni sono file JSON versionabili con Git, gli ambienti gestiscono variabili e segreti, e lo stesso motore che gira nell'app esegue le suite in CI da riga di comando.

Perché Rustman



Workflow a nodi

Costruisci catene di chiamate su un canvas: rami condizionali sugli archi, cattura di variabili e merge, con esecuzione a grafo.



Dashboard

Home con KPI del workspace, attività recente e distribuzione dei metodi HTTP a colpo d'occhio.



Git integrato

Commit, diff, cronologia e remote con pull/push direttamente sul workspace delle collezioni.



Test & contract

Asserzioni su status/JSON/JSONPath, JSON Schema, snapshot/golden testing e script `pm.*`.



Performance

Test di carico a durata/RPS con warmup, grafici e gate SLO su p95 ed error-rate.



Code-gen multi-lingua

Genera lo snippet della richiesta in cURL, fetch (JS), Python, **Java** e **C#**.



GraphQL · WS · SSE

Editor GraphQL con introspezione, console WebSocket e stream di eventi Server-Sent Events.

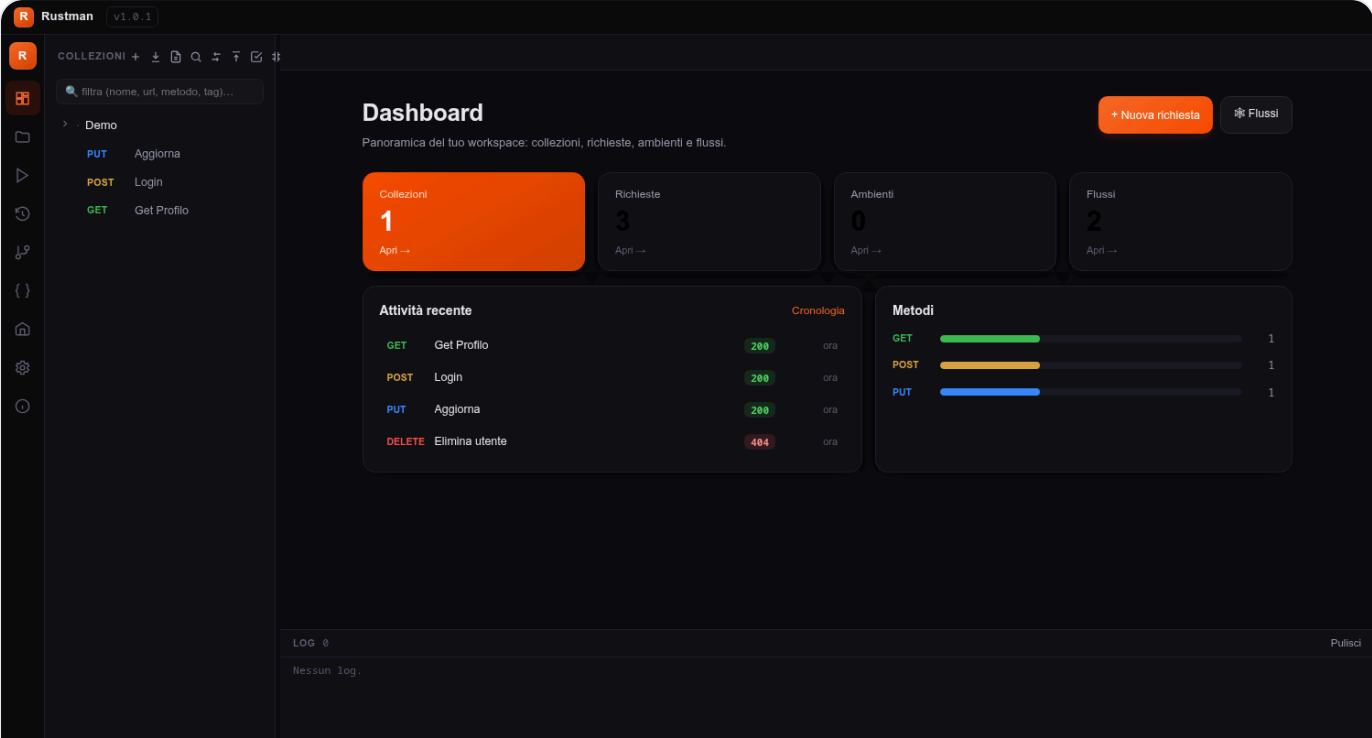


Import / export

Postman, OpenAPI/Swagger, cURL e HAR. Mock server e generatore di documentazione HTML.

Tutto il workspace a colpo d'occhio

La Home riassume collezioni, richieste, ambienti e flussi, con l'attività recente e la distribuzione dei metodi. L'accento è personalizzabile a runtime (Rust, Verde, Viola o un colore a scelta).



Dashboard. KPI del workspace, attività recente con esito e barre per metodo HTTP.

Guida completa a Rustman

Concetti, esempi e diagrammi per usare ogni funzione del software.

1 Architettura

2 Workspace, collezioni, richieste

3 Ambienti e variabili

4 Anatomia di una richiesta

5 Generazione codice

6 Il canvas dei flussi

7 Esecuzione e risultati

8 Test e asserzioni

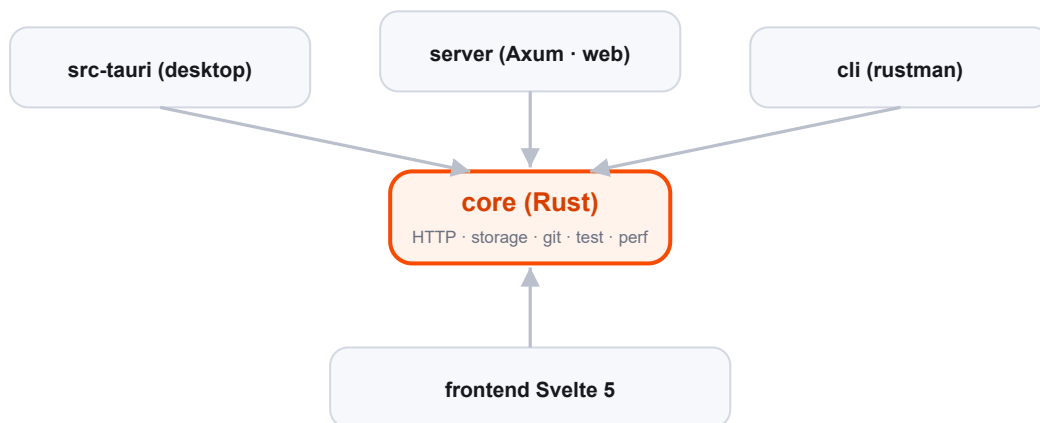
9 CLI & CI

10 Altre funzioni

01

Architettura

Rustman è un monorepo: la logica vive in un core Rust riutilizzato da tre "gusci" (desktop, web, CLI). Il frontend Svelte è identico su desktop e web — cambia solo il trasporto verso il backend.



Il **core** è condiviso: la stessa logica alimenta app desktop, server web e CLI. Il frontend parla al backend via comandi Tauri (desktop) o REST **/api** (web).

02

Workspace, collezioni, richieste

Un **workspace** è una cartella. Ogni **collezione** è una sottocartella con **richieste** salvate come file **.json** (anche in cartelle annidate). Essendo file di testo, tutto è versionabile con Git.

```
# struttura tipica di un workspace
workspace/
  Anvil/                # una collezione
    crea-progetto.json
    cerca-progetti.json
  environments/         # ambienti
    anvil.json
  runs/                 # flussi (workflow)
    anvil-crea-verifica.json
  .rustman-secrets.json # segreti, gitignorati
```

Segreti al sicuro. Le variabili marcate come segrete non finiscono nei file committati: vengono salvate a parte in `.rustman-secrets.json`, che è gitignorato.

03

Ambienti e variabili

Le variabili si richiamano con la sintassi `{{nome}}` in URL, header, query e body. Un ambiente (es. *staging*, *prod*) definisce i valori; puoi cambiarlo al volo. Esistono anche variabili **dinamiche** e **faker**.

TIPO	ESEMPI	USO
Ambiente	<code>{{baseUrl}}</code> · <code>{{apiKey}}</code>	Valori per-ambiente, con segreti
Dinamiche	<code>{{timestamp}}</code> · <code>{{randomUUID}}</code> · <code>{{randomInt}}</code>	Generate all'invio
Faker	<code>{{name}}</code> · <code>{{email}}</code>	Dati finti realistici
Catturate	<code>{{projectId}}</code>	Estrate dalle risposte nei flussi

04

Anatomia di una richiesta

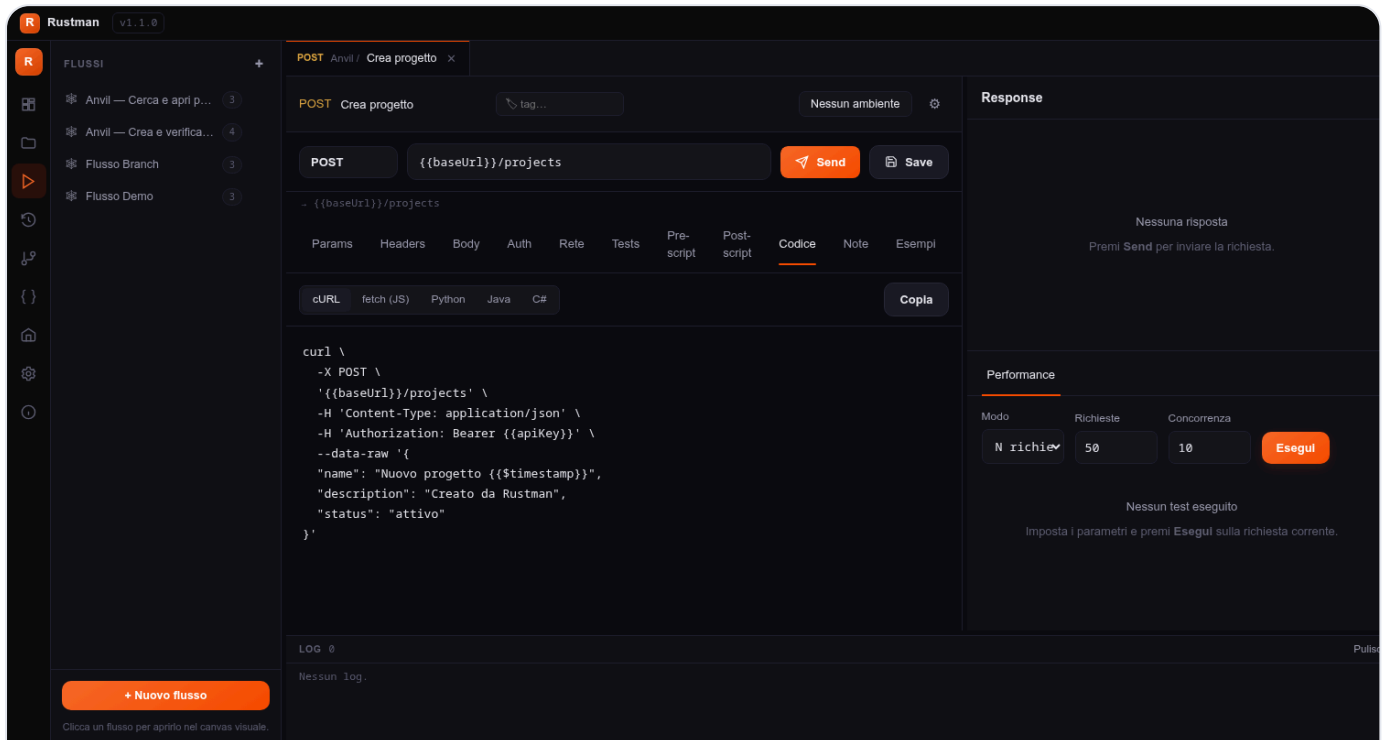
L'editor organizza tutto in schede: **Params**, **Headers**, **Body** (raw, form-data con upload, x-www-form-urlencoded), **Auth** (Bearer, Basic, OAuth 2.0), **Rete** (timeout, redirect, TLS, retry 429), **Tests**, **Pre/Post-script**, **Codice**, **Note** ed **Esempi**.



Ciclo di vita di un passo: lo script pre può modificare richiesta e variabili; dopo la risposta si valutano test e script post, poi le catture salvano valori in variabili.

Generazione codice

La scheda **Codice** mostra la richiesta corrente tradotta in cinque linguaggi, pronta da copiare. Header, autenticazione, query e body vengono riportati fedelmente.



Scheda Codice. Selettore lingua (cURL · fetch · Python · Java · C#), anteprima e copia con un click.

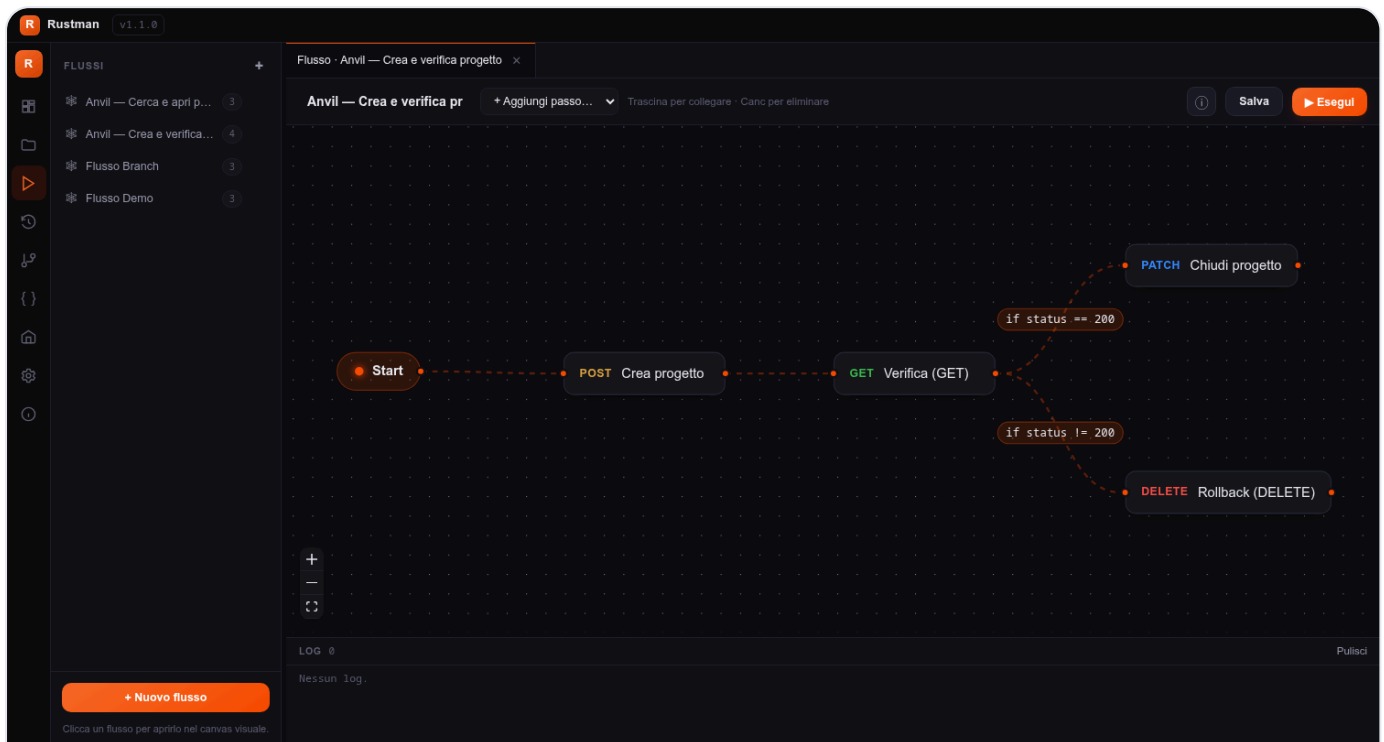
Esempio — la stessa richiesta in Java e C#

```
// Java 11+ (java.net.http)
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("{{baseUrl}}/projects"))
    .method("POST", HttpRequest.BodyPublishers.ofString(body))
    .header("Authorization", "Bearer {{apiKey}}")
    .build();
```

```
// C# (System.Net.Http)
var request = new HttpRequestMessage(new HttpMethod("POST"), "{{baseUrl}}/projects");
request.Content = new StringContent(body);
request.Headers.TryAddWithoutValidation("Authorization", "Bearer {{apiKey}}");
```

Il canvas dei flussi

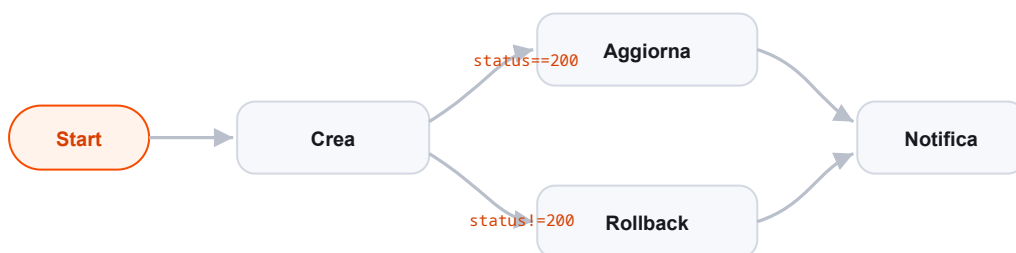
Un **flusso** è un test d'integrazione visuale: una catena di richieste disegnata come grafo. Si apre nel canvas dalla sezione **FLUSSI**; ogni nodo è una richiesta della collezione, collegata da archi.



Canvas. Da *Start* si esegue lungo gli archi. Qui un ramo prosegue se `status == 200`, l'altro fa il rollback.

Rami condizionali e merge

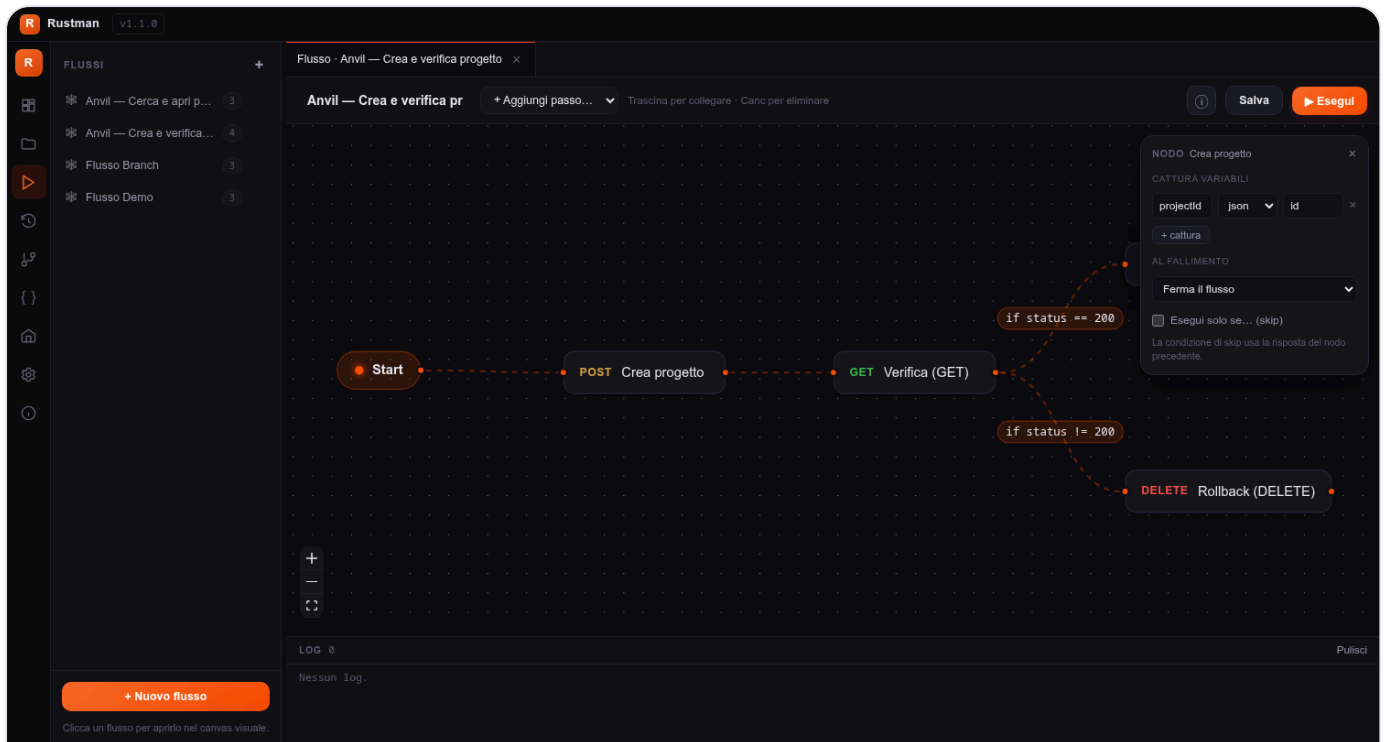
Cliccando un **arco** puoi imporgli una condizione (su `status`, un campo `json` o una `var`): l'arco viene percorso solo se è vera. Più archi in uscita = biforcazione; più archi in entrata sullo stesso nodo = merge (il nodo gira una sola volta).



Branch sulle uscite di *Crea*, merge sulle entrate di *Notifica*. L'esecuzione segue un ordine topologico e i rami non presi risultano **SKIP**.

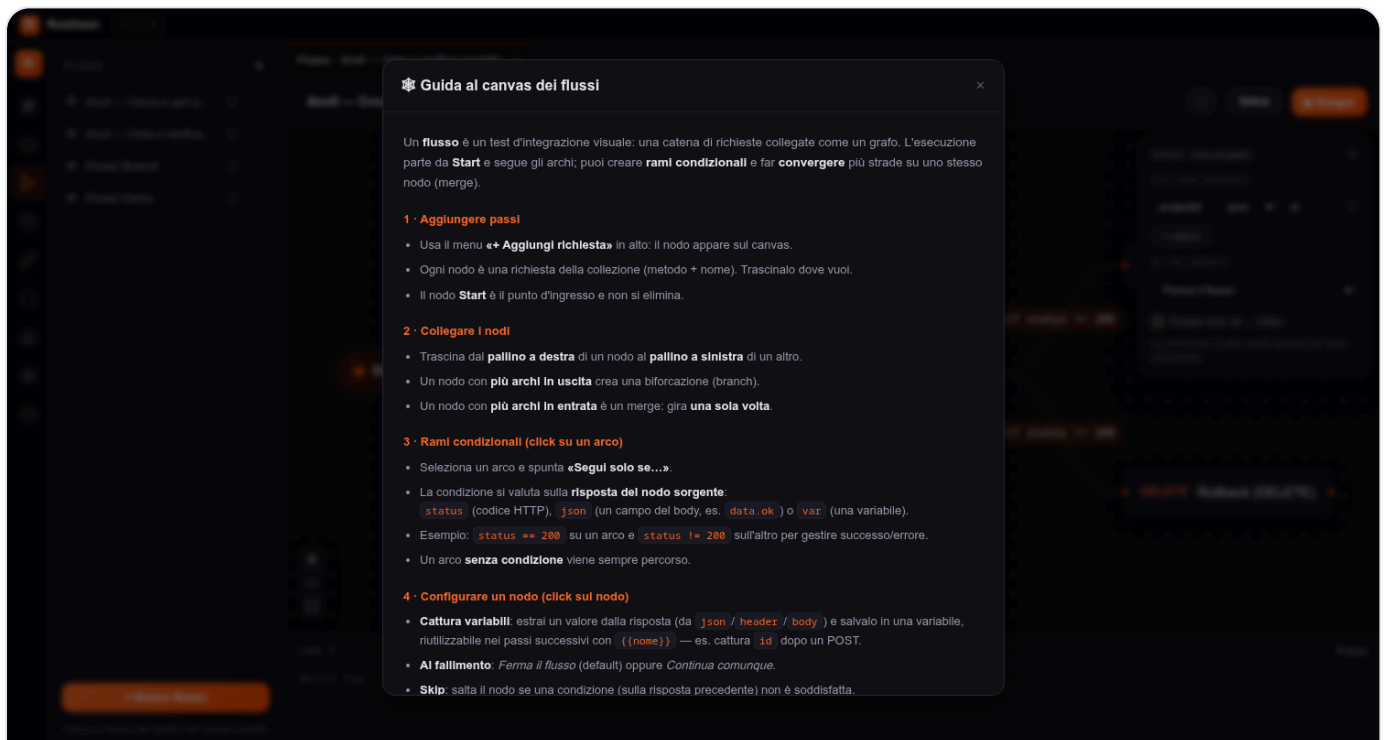
Inspector di nodo

Cliccando un **nodo** si apre il pannello di configurazione: **cattura variabili** (estrai un valore dalla risposta e salvalo, es. `projectId ← json.id`), comportamento **al fallimento** (ferma / continua) e **skip** condizionale.



Inspector di nodo. Catture, gestione del fallimento e condizione di skip — tutto sul canvas.

Guida integrata. Il pulsante ⓘ in alto apre una guida passo-passo direttamente nel canvas.

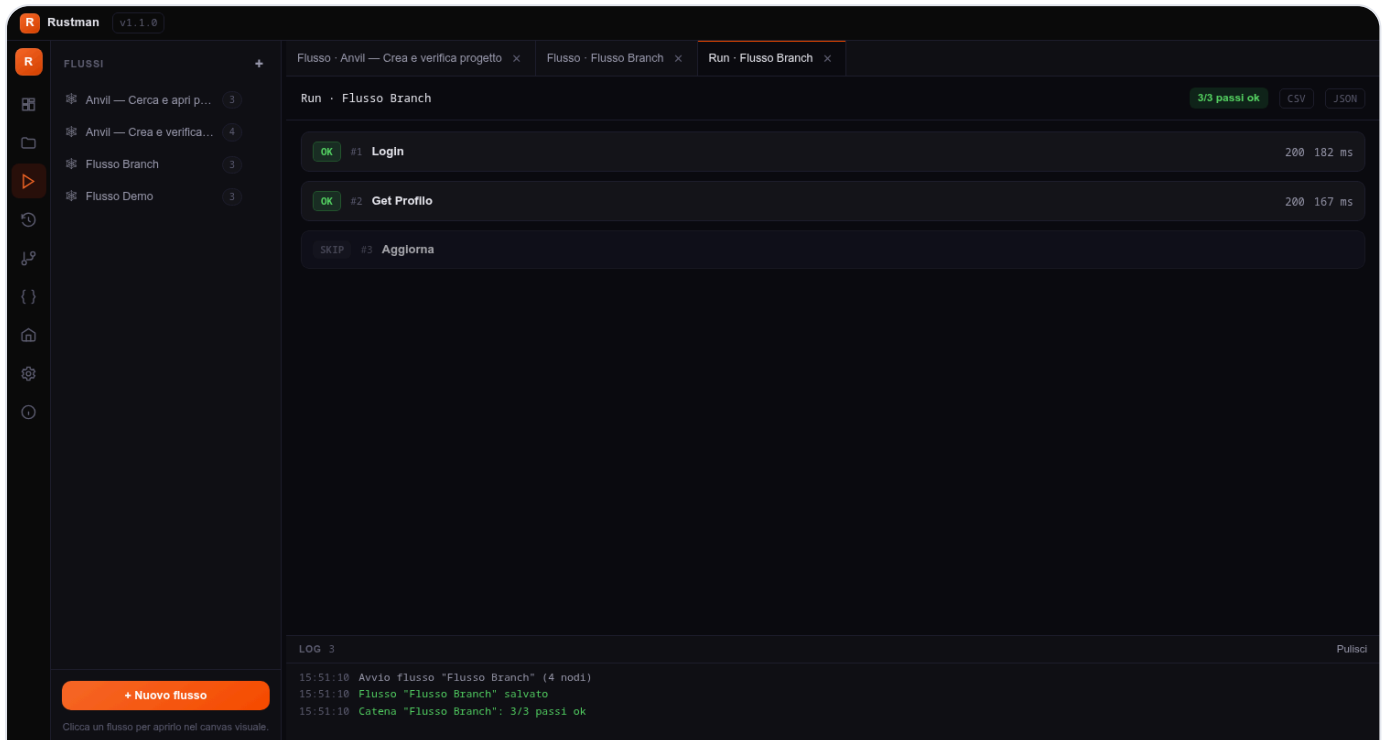


Guida del canvas. Aggiungere passi, collegare, rami, configurazione nodo, esecuzione e scorciatoio.

07

Esecuzione e risultati

Il pulsante ► **Esegui** salva e lancia il flusso; i risultati si aprono in una scheda dedicata, passo per passo, con status, tempo e test. I nodi non raggiunti (rami non presi) sono marcati **SKIP**.



Risultati. Esito per passo con status e tempi; export in CSV/JSON. Qui un ramo è stato saltato correttamente.

08

Test e asserzioni

Ogni richiesta può avere asserzioni dichiarative (status, tempo, header, body, JSON, JSONPath, JSON Schema) e script `pm.*` in stile Postman. Le stesse verifiche girano nella CLI.

```
// Post-script: salva il token e verifica lo status
const d = pm.response.json();
pm.environment.set("token", d.token);
pm.test("status 200", () => pm.expect(pm.response.code).to.equal(200));
```

- ✓ Asserzioni `schema` (JSON Schema), inferibili dalla risposta o dall'import OpenAPI
- ✓ Snapshot / golden testing con ignore-paths e approvazione della baseline
- ✓ Security scan delle risposte: header di sicurezza, CORS, info leak, cookie

09

CLI & CI

L'eseguitore headless `rustman run` lancia collezioni e flussi, valuta le asserzioni ed esce con codice $\neq 0$ al fallimento. Lo stesso motore a grafo dell'app vale in pipeline.

```
# esegui un flusso a nodi con report JUnit
rustman run ./workspace --chain "Anvil – Crea e verifica progetto" --junit report.xml

# test di carico con gate SLO (fallisce se p95 o error-rate sfiorano)
rustman perf ./workspace --request Anvil/lista-progetti.json \
  --duration 30 --concurrency 20 --rps 50 --max-p95 200 --max-error 1

# copertura delle API rispetto a uno spec OpenAPI
rustman coverage ./workspace --spec openapi.yaml
```

- ✓ Run **data-driven** (`--data` CSV/JSON): un'iterazione per riga
- ✓ Matrice di ambienti (`--envs staging,prod`) e gate del pass-rate
- ✓ Mock server dallo spec OpenAPI o dalle snapshot registrate

10

Altre funzioni

GraphQL

Editor query + variabili, esecuzione ed explorer dello schema via introspezione.

WebSocket / SSE

Console interattiva per inviare/ricevere messaggi e seguire stream di eventi live.

Import

Postman (Collection v2 + Environment), OpenAPI/Swagger 3.x e 2.0, cURL e HAR.

Documentazione

Generatore HTML dalle collezioni, con esempi di risposta salvati per richiesta.

History & diff

Cronologia con replay, confronto fra due risposte e trend dei tempi per endpoint.

Temi & accento

Chiaro/scuro/sistema, dimensione font e colore d'accento personalizzabile a runtime.

Provalo. Scarica l'ultima release per il tuo sistema da github.com/russus87/rustman/releases — Linux, Windows e macOS. Open source, licenza MIT.